National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 8: Object Detection using Faster R-CNN

Custom Model Implementation on Stop Signs

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 12, 2026

1 Objectives

The primary objective of this tutorial is to implement a custom object detection model using the **Faster R-CNN architecture**. Key learning outcomes include:

- Understanding the Faster R-CNN architecture, specifically the **Region Proposal Network (RPN)**.
- Understanding **Region of Interest (ROI) pooling**.
- Calculating **Intersection over Union (IoU)** for bounding box accuracy.
- Evaluating model performance using **Mean Average Precision (mAP)**.
- Transitioning from a theoretical Keras framework to a **PyTorch** implementation using a custom dataset.

2 Dataset Preparation

Instead of relying on a pre-packaged academic dataset, a custom dataset was curated specifically for this task to detect **Stop Signs**.

2.1 Data Collection & Annotation

A total of **125 images** featuring stop signs in diverse lighting conditions, angles, and environments were scraped and aggregated. The dataset was uploaded to **Roboflow,** where manual bounding box annotations were drawn to establish the ground truth.

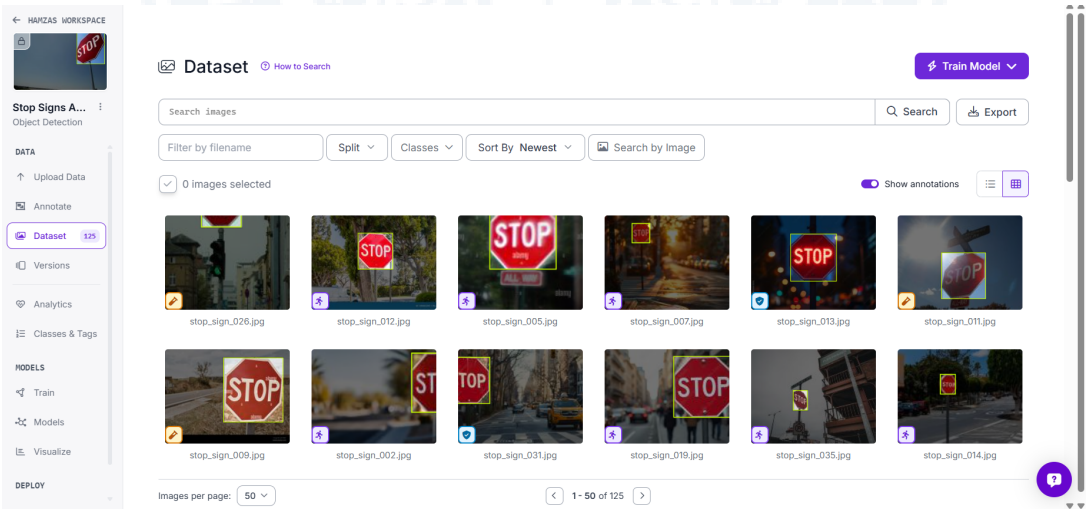

Figure 1: Roboflow Website Dataset.

2.2 Data Splitting & Coordinate Formatting

The dataset was split into training, validation, and testing subsets. The annotations were exported in YOLO format (normalized center coordinates) and had to be dynamically converted to PyTorch's Faster R-CNN format (absolute corner coordinates: $[x_{min}, y_{min}, x_{max}, y_{max}]$).

A custom `torch.utils.data.Dataset` class was engineered to handle this mathematical conversion on the fly during training:

```
    # YOLO format: cls, x_center, y_center, width, height (normalized 0-1)
    # Faster R-CNN requires absolute pixel coordinates: [xmin, ymin, xmax, ymax]

    xmin = (x_center - width / 2) * image_width
    xmax = (x_center + width / 2) * image_width
    ymin = (y_center - height / 2) * image_height
    ymax = (y_center + height / 2) * image_height

    boxes.append([xmin, ymin, xmax, ymax])

```

Listing 1: Custom PyTorch script converting normalized YOLO coordinates to absolute Faster R-CNN bounding boxes.

3 Methodology: Faster R-CNN Architecture

This implementation utilizes PyTorch's `torchvision` library. The model consists of two primary stages:

3.1 Feature Extraction and RPN

A pre-trained **ResNet-50** backbone, coupled with a **Feature Pyramid Network (FPN)**, was used for robust multi-scale feature extraction. The **Region Proposal Network (RPN)** slides over these feature maps to predict "objectness" scores and bounding box refinements (anchors), effectively answering: *"Is there an object here, and roughly where is it?"*

3.2 ROI Pooling and Classification

The regions proposed by the RPN are varying in size. **ROI Pooling** extracts fixed-size feature maps from these uneven proposals. Finally, the pre-trained classification head was replaced with a new `FastRCNNPredictor` configured for exactly **2 classes**: Class 0 (Background) and Class 1 (Stop Sign).

4 Model Training

The custom model was moved to a **CUDA-enabled GPU (T4)** for accelerated training. PyTorch dynamically handles the summation of the four distinct Faster R-CNN losses during training (classification loss, box regression loss, objectness loss, and RPN box loss).

4.1 Hyperparameter Configuration

Optimization was performed using **Stochastic Gradient Descent (SGD)**. The specific hyperparameters utilized to achieve stable convergence are detailed in Table 1.

Table 1: Training Hyperparameters

Hyperparameter	**Value**
Optimizer	SGD
Learning Rate	0.005
Momentum	0.9
Weight Decay	0.0005
Batch Size	4
Epochs	10

4.2 Training Loss Analysis

As demonstrated in Table 2 and visualized in Figure 2, the model exhibited rapid learning during the initial epochs. The steep gradient between Epoch 1 and Epoch 3 indicates the RPN quickly learned to identify regions of interest, while the gradual plateau toward a **final loss of 0.3654** suggests stable convergence without aggressive over-fitting.

Table 2: Training Loss per Epoch

Epoch	**Total Loss**
1	5.3170
2	1.4793
3	0.9664
4	0.6847
5	0.5917
6	0.5152
7	0.4828
8	0.4047
9	0.4034
10	**0.3654**

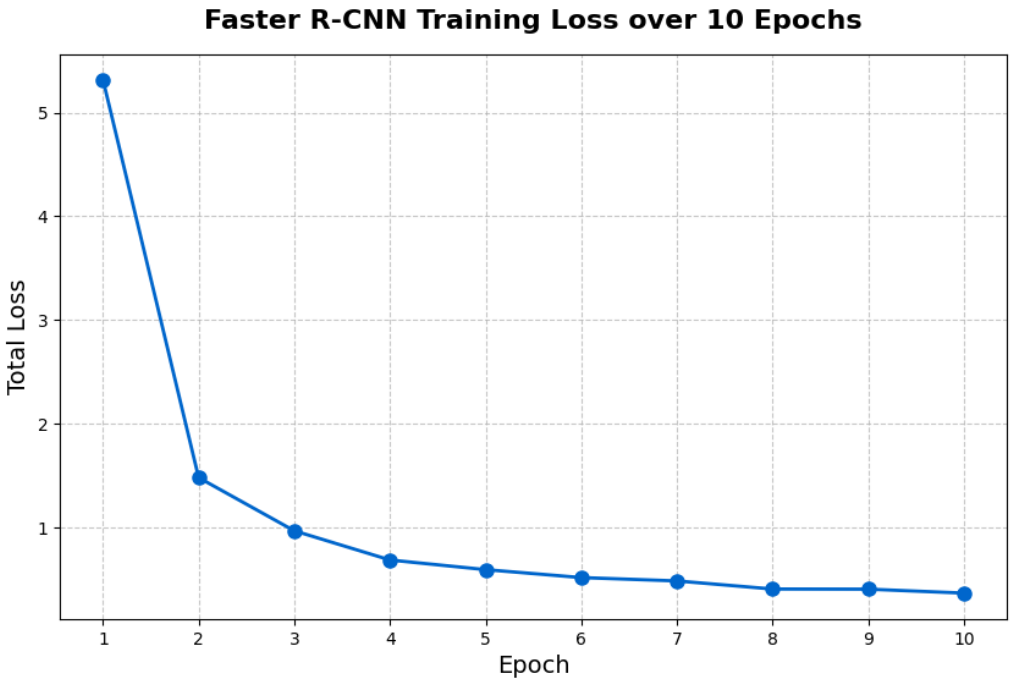

Figure 2: Convergence of the Faster R-CNN model over 10 training epochs.

5 Evaluation and Results

5.1 Mean Average Precision (mAP)

Average Precision (AP) represents the area under the Precision-Recall (PR) curve and is defined mathematically as:

$$AP = \int_0^1 P(R)dR \tag{1}$$

The Mean Average Precision (mAP) is the average of the AP calculated across all classes. The model was evaluated on 12 unseen test images, yielding excellent results:

- **mAP @ 0.50 (IoU threshold 50%): 0.88**
- **mAP @ 0.50:0.95 (Strict IoU thresholds): 0.74**

5.2 Discussion of Performance

Achieving an **mAP of 0.88** is a highly significant result for a custom object detector trained from scratch on only 125 images. This high precision indicates that the **ResNet-50 backbone** successfully transferred its generalized feature extraction capabilities to the specific spatial geometries of stop signs. Furthermore, the strict mAP of **0.74** proves that the bounding box regressions are exceptionally tight, localizing the object with minimal background noise.

5.3 Visual Inference

To visually verify the metrics, the model was set to evaluation mode and tested on a grid of random, unseen images. Using a confidence threshold of 50%, the model success-

fully localized the stop signs in complex environments, drawing accurate bounding boxes despite varied angles and lighting conditions (Figure 3).

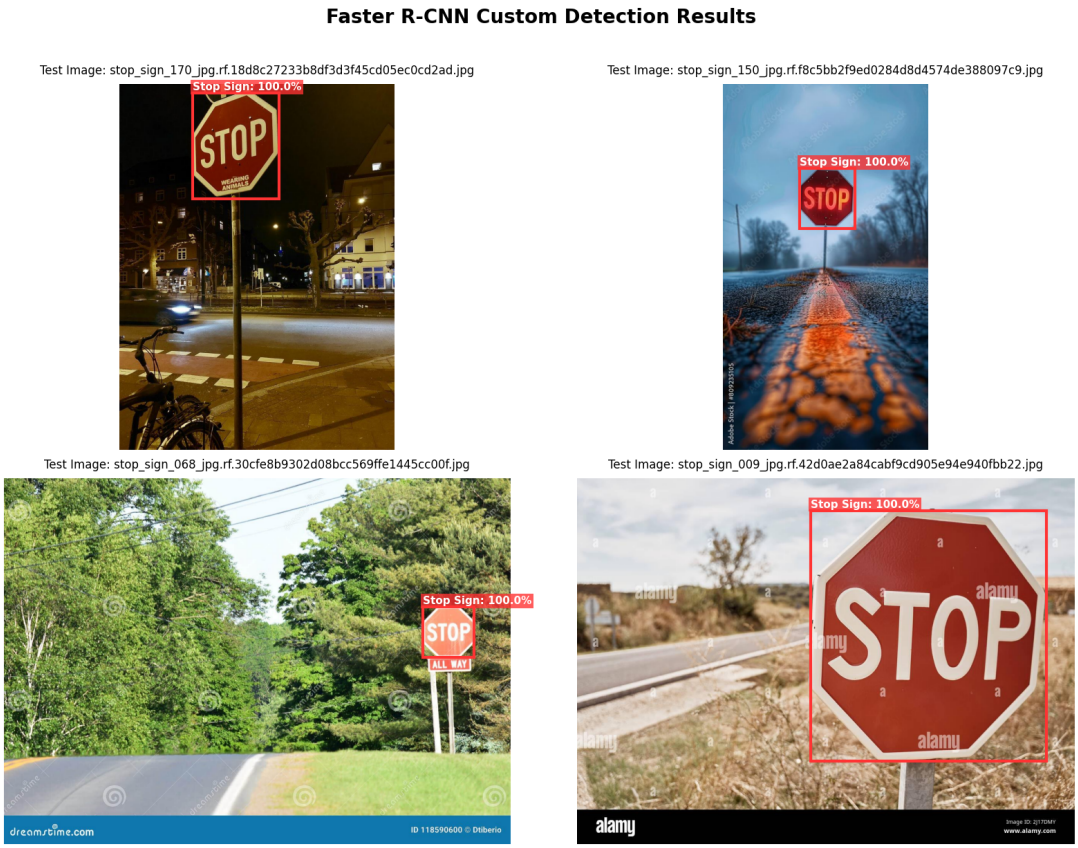

Figure 3: Inference results on unseen test images showing bounding box predictions and confidence scores.

6 Conclusion

This tutorial successfully demonstrated the end-to-end pipeline of building a custom object detector. By manually annotating a unique dataset, programming a custom PyTorch data loader for coordinate transformation, and fine-tuning a pre-trained **Faster R-CNN ResNet-50** model, the system achieved a highly accurate detection rate for stop signs. The final training loss converged smoothly to **0.3654**, culminating in a robust **mAP of 0.88**.